

Reporte de implementación.

Mauricio Olguín Sánchez - A01711522

26 de agosto de 2026

Índice general

1. Realizando la implementación «A manita»

- 1.1. Sobre la base de datos seleccionada
- 1.2. Proceso de ETL
 - 1.2.1. *Extraction*
 - 1.2.2. *Transforming*
 - 1.2.3. *Loading*
- 1.3. *Exploratory Data Analysis*

2. Anexo

- 2.1. Figuras amplias
- 2.2. Códigos

Bibliografía

Capítulo 1

Realizando la implementación «Amanita»

1.1. Sobre la base de datos seleccionada

Después de haber investigado en páginas como [UC Irvine](#), [Kaggle](#), el [World Bank Open Data](#), entre otros, encontré en la página de [OpenML](#) un *dataset* interesante. Tras aplicar varios filtros, encontré la base de datos [Airlines_DepDelay_10M](#).

Esta base de datos está disponible en OpenML y está construida a partir de datos provenientes de la Oficina de Estadísticas de Transporte de los Estados Unidos (*Bureau of Transportation Statistics*, BTS), específicamente de la base de datos *Airline On-Time Performance Data*.

La base de datos seleccionada contiene 10,000,000 de registros y 10 variables, y corresponde a un problema de regresión. A continuación, presento una breve descripción general del *dataset*:

Propósito: Predecir el tiempo de retraso en la salida de los vuelos, medido en minutos (*Departure Delay*).

Fecha de los datos: Desde 1987 hasta 2013.

Formato: ARFF (*Attribute-Relation File Format*).

Tipo de tarea: Regresión.

Número de registros: 10,000,000.

Número de variables: 10.

Aunque OpenML proporciona información sobre cada variable, dentro de la fuente original (en el *Bureau of Transportation Statistics*), existe una descripción más detallada sobre los datos:

DepDelay - numerico. Diferencia, en minutos, entre la hora de salida programada y la hora de salida real del vuelo. Los valores negativos indican que el vuelo salió antes de lo programado.

Month - numerico. Mes en el que se realizó el vuelo, con valores del 1 al 12, donde 1 corresponde a enero y 12 a diciembre.

DayofMonth - numerico. Día del mes en el que se realizó el vuelo, con valores del 1 al 31.

DayOfWeek - numerico. Día de la semana en el que se realizó el vuelo, representado mediante valores del 1 al 7.

CRSDepTime - numerico. Hora de salida programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

CRSArrTime - numerico. Hora de llegada programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

1.2. PROCESO DE ETL

UniqueCarrier - categorico. Código único utilizado para identificar a la aerolínea que opera el vuelo.

Origin - categorico. Código del aeropuerto de origen del vuelo.

Dest - categorico. Código del aeropuerto de destino del vuelo.

Distance - numerico. Distancia total del vuelo, expresada en millas.

En contraste, las variables *UniqueCarrier*, *Origin* y *Dest* son explícitamente categóricas, pero serán excluidas del modelo. Su transformación (utilizando la misma técnica) produciría un incremento considerablemente mayor en la cantidad de variables. Principalmente, tomo esta decisión con el objetivo de mantener un tiempo de ejecución razonable y centrar el proyecto en la implementación manual del algoritmo de regresión.

1.2. Proceso de ETL

1.2.1. *Extraction*

Como ya lo he mencionado antes, los datos se han obtenido del siguiente link: <https://www.openml.org/search?type=data&status=active&id=42728>

Sin embargo, si se quisiera obtener más datos actuales, habría que descargarlos (sea de forma manual o creando algún *script*) desde el siguiente link: <https://transtats.bts.gov/PREZIP/>.

Aquí, se recopilan todos los datos desde 1987 hasta junio de 2026.

1.2.2. *Transforming*

Las variables *Month*, *DayofMonth* y *DayOfWeek*, aunque se encuentran representadas numéricamente en el conjunto de datos, las trataré como variables categóricas. Esto se debe a que, en realidad, los valores numéricos representan categorías y no implican necesariamente una relación lineal entre ellas. Por ejemplo, el mes representado por el valor 12 no debe interpretarse como una categoría doce veces mayor que el mes representado por el valor 1.

Para representar estas variables dentro del modelo de regresión, utilizaré la técnica de *One-Hot Encoding*. Aunque esto implicará que crezca la dimensionalidad del conjunto de datos.

Sobre los atributos *CRSDepTime* y *CRSArrTime*

Las variables *CRSDepTime* y *CRSArrTime* representan las horas programadas de salida y llegada de los vuelos. En el conjunto de datos, estas se representan en el formato HHMM, es decir, que si la hora era 13:20, en los datos se muestran 1320.

Sin embargo, esta representación no resulta adecuada al momento de utilizarla directamente como valores numéricos porque, dentro del modelo de regresión, no será representado la distancia real. Por ejemplo, los valores de 2355 y 5, aunque están apartados por 10 minutos, utilizando los datos tal cual como vienen en el conjunto de datos, estos estarían muy separados entre sí.

Es por esto que ambas variables tienen que ser transformadas y, la mejor conversión que he podido investigar es utilizar como referencia la medianoche. Para realizar esta conversión, se separó la hora y los minutos del valor original y se aplicó la siguiente expresión:

$$\text{Minutos}_{\text{desde la media noche}} = 60 \times \text{Hora} + \text{Minutos}$$

De esta forma, conservamos la información temporal original y proporcionamos una representación numérica más adecuada para el modelo de regresión.

1.3. EXPLORATORY DATA ANALYSIS

Sobre el formato del archivo de los datos

Al empezar a realizar una exploración de los datos, cada que tenía que ejecutar el programa para visualizarlos, tardaba un aproximado de 45 segundos cada que ejecutaba el programa. Investigando opciones para solucionar este problema, me encontré con el formato parquet. Este formato está precesamente optimizado para mejorar el manejo de datos complejos, el rendimiento de las consultas que se hacen y la eficiencia del almacenamiento [1].

Esta integración, al ver que me resultaría óptima para poder realizar más pruebas más rápidas, la he implementado. Así pues, el archivo ha pasado de tener un peso de 385 MB a sólo 56 MB; además de que el tiempo de carga se redujo drásticamente de 45 segundos a 2 segundos o menos.

1.2.3. Loading

Una vez hechas las transformaciones anteriores, ahora dividiré el tamaño de mi *dataset* en tres principales secciones: *train*, *validation* y *test*. Al ser la cantidad de datos tan alta, podría realizar varias particiones para *validation*, sin embargo, en esta primera parte en donde me enfoco aprender el algoritmo, sólo haré una vez esta división. Así pues, utilizaré una proporción de 98 % de los datos para *train*, 1 % para *validation* y 1 % para *test*. [2]

1.3. Exploratory Data Analysis

Hice un primer acercamiento general a los datos simplemente realizando un gráfico de pares (*pairplot*), el cuál resultó en el siguiente: (ver figura 1.1)

Como era de esperarse, al tener datos que corresponden a periodos de tiempo, realmente no podemos ver una tendencia o información relevante de estos. Sin embargo, vale la pena destacar las comparaciones

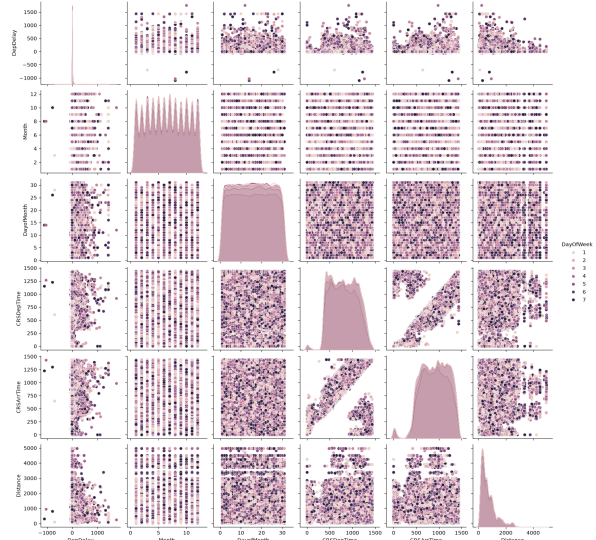


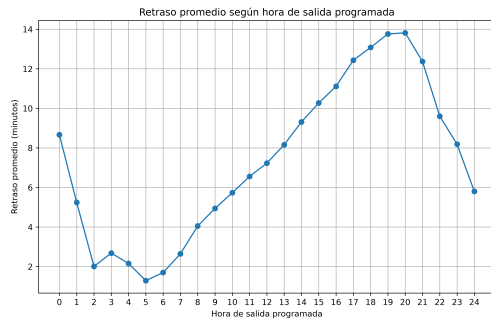
Figura 1.1: *Pair plot* para visualizar la mayoría de los datos comparados entre sí. No se graficaron todos los 10 millones de puntos por el tiempo de computo requerido, además de que tampoco es la intención de este gráfico. Se añadió también que, para cada punto, se etiquetara por su el día de la semana (*DayOfTheWeek*) para intentar de visualizar algún *cluster* o alguna zona de interes.

cuando los datos no se están comparando entre dos periodos de tiempo (con excepción de *CRSDepTime* vs *CRSArrTime*). También, no podemos observar alguna zona en donde los datos se acumulen por algún periodo de tiempo.

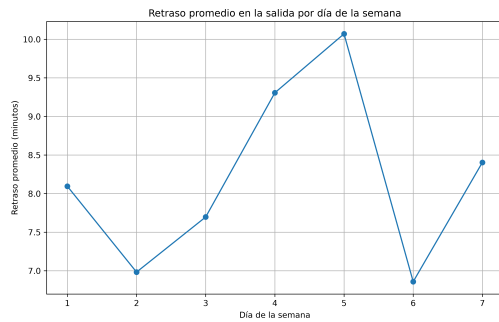
Aunque antes he hecho el proceso de hacer un *One Hot Encoding* para las variables categóricas, sirve también visualizar de forma directa estos datos tal cual dentro de estas gráficas para darme una idea general de los datos.

Luego, teniendo en cuenta que cuento con los datos de los días de la semana, días del mes, meses del año y las horas en las que se retrasan, decidí hacer un gráfico con los minutos promedio por cada serie de tiempo para ver cómo es que evolucionan los retrasos durante cada periodo de tiempo.

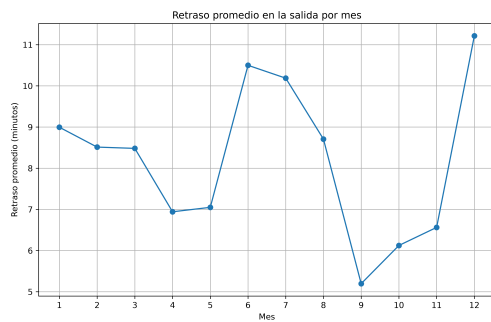
1.3. EXPLORATORY DATA ANALYSIS



(a) Promedios de retrasos por cada hora.



(b) Promedios de retrasos por cada día de la semana.



(c) Promedios de retrasos por cada mes.

Figura 1.2: Promedios de retrasos por periodos de tiempos.

Capítulo 2

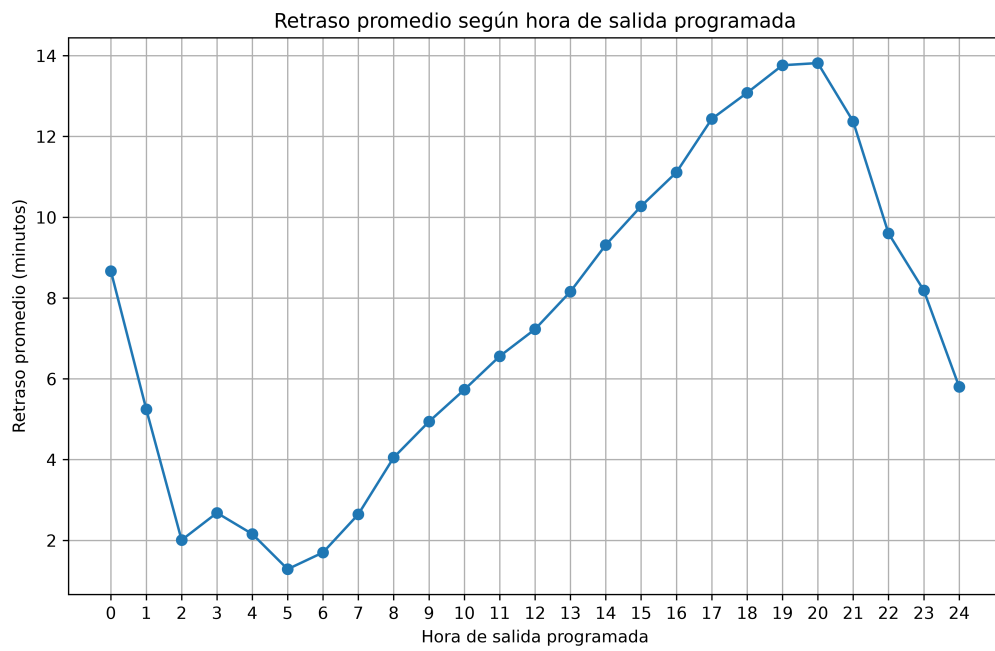
Anexo

2.1. Figuras amplias

A continuación, se vuelven a dejar todas las figuras que contienen gráficos en un mayor tamaño poder visualizarlos mejor.



2.1. FIGURAS AMPLIAS



2.2. CÓDIGOS



2.2. Códigos

El siguiente código sirve únicamente para la generación de los gráficos presentados, aunque no todos fueron utilizados.

Código principal.

Bibliografía

- [1] Matthew Kosinski Annie Badman. *¿Qué es Apache Parquet? / IBM* — *ibm.com*. <https://www.ibm.com/mx-es/think/topics/parquet>. [Accessed 26-08-2026].
- [2] scgupta. *Training, Validation, and Test Split for ML Models* — *ml4devs.com*. <https://www.ml4devs.com/what-is/train-validation-test-data-split/>. [Accessed 26-08-2026].